

Huanhuan's Notes

Visibility: Confidential ▾

Status: **Draft** | In Review | Approved | Final | Obsolete

Author(s): Huanhuan Liu

Email: huanhl@hotmail.com

Version: v1.0.2

File Creation Date: 2016-01-16

Updated: 2026-04-17

Introduction

This section is dedicated to documenting personal experiences, insights, and learnings. By consistently recording our thoughts and reflections, we create opportunities for self-improvement and growth.

Please note that this document is a draft and may contain errors. Users are encouraged to verify information and make necessary corrections.

Background / Motivation

Poor Handwriting is Better Than a Good Memory.

Table of contents

[Introduction](#)

[Background / Motivation](#)

[Table of contents](#)

[Prepare](#)

[Environment](#)

[Linux Distribution \(Optional\)](#)

[Ubuntu](#)

[Download Ubuntu Desktop](#)

[Install OS](#)

[Debian \(13 Trixie - stable channel\)](#)

[Download Debian Live CD](#)

[Install OS](#)

[Install Package](#)

[Install Software](#)

[Configure System Settings](#)

[Install Chinese Input](#)

[Install VPN for Linux - Optional](#)

[Clash Party \(Recommand\)](#)

[Open VPN Connect](#)

[WireGuard](#)

[Terminal customization](#)

[Starship](#)

[Installation](#)

[Configuration](#)

[Apply Changes](#)

[Zsh and Powerlevel10k \(Optional, Not recommend\)](#)

[Prerequisites](#)

[Run the zsh installation script](#)

[Install Powerline fronts](#)

[Install Powerlevel10k \(p10k\)](#)

[Setup theme to p10k](#)

[Apply Changes](#)

[Configure](#)

[Git and Repo](#)

[Install Git](#)

[Install Repo](#)

[Ubuntu](#)

[Debian](#)

[Configure git](#)

[Setup git credential](#)

[Docker](#)

[Official Reference](#)

[Ubuntu](#)

[Debian](#)

[Install using the apt repository](#)

[Set up Docker's apt repository](#)

[Install the Docker packages](#)

[Verify that the installation is successful by running the hello-world image](#)

[Setup permissions for docker and tty Users](#)

[Factory Server](#)

[Installation](#)

[Get deployment script](#)

[Download and install factory server](#)

[Update deployment script and server images](#)

[Build Factory Server](#)

[Build the dev version factory server](#)

[Run the dev version factory server](#)

[Debug](#)

[Access](#)

[Fastboot](#)

[Important commands](#)

[Flash boot partition](#)

[Flash all partitions in \\$ANDROID_PRODUCT_OUT defined in fastboot-info.txt](#)

[Flash MBR-GPT binary](#)

[Get key variables](#)

[Get FW variant or board name \(depending on fastboot mode\)](#)

[Reboot to bootloader mode \(has to be executed in userspace fastboot\)](#)

[Reboot to userspace fastboot mode \(has to be executed in FW fastboot\)](#)

[Reboot device](#)

[Set active slot as a](#)

[Check current FW mode](#)

[Apart from disk provisioning](#)

[Change selinux](#)

[Enable AP console](#)

[Set successful boot flag](#)

[Get kernel boot priorities](#)

[Fastboot with DOME Server](#)

[Fastboot with Fastboot Orchestrator](#)

[Factory Server Development](#)

[Network Configuration for Factory Server](#)

[Recommended Method: Using NetworkManager \(shared\)](#)

[Legacy Netboot/DHCP Setup \(Deprecated\)](#)

[Manual dnsmasq Setup \(Deprecated\)](#)

[TFTP Configuration \(Deprecated\)](#)

[Setup TFTP server and DHCP server with dnsmasq](#)

[Install dnsmasq](#)

[Check network interface](#)

[Configure network interface](#)

[Apply settings](#)

[Decide a folder to put TFTP files, for example, /var/tftp](#)

- [Setup tftp Permission](#)
- [Configure dnsmasq \(DHCP + TFTP\)](#)
- [Check dnsmasq config file](#)
- [Check and start dnsmasq](#)
- [Manually run dnsmasq {Debug}](#)

[Setup DOME server for fastboot](#)

- [Fastboot with DOME / UMPIRE server](#)

[Debug Tips](#)

- [Enter umpire shell](#)

[Aluminium OS](#)

[Context](#)

- [Objective](#)

- [Background](#)

[Prerequisites](#)

- [System Requirements](#)

- [Install Required Packages](#)

- [Account Registration](#)

- [Generate git cookies](#)

[Get the Source Code](#)

- [Branch Build - ARSP 20260417 update](#)

[Images](#)

[Build Images](#)

- [Tips: Reset git commit head](#)

- [Tips: Install the Google Cloud CLI](#)

[Building Kernel](#)

- [Using Kernel Prebuilts for Android image](#)

- [Using Locally Built Kernel as Replacement \(Optional\)](#)

- [Fetch from ARSP](#)

- [Using Locally Built Kernel as Replacement with version'd manifest \(Optional\)](#)

- [Fetch from ARSP](#)

- [Build and replace kernel](#)

[Make Pre-flash Image](#)

- [Tips: Install lpadd lpdump lpunpack](#)

- [Tips: Debug file mtime error](#)

[Flashing Aluminium](#)

- [Disable WP \(write-protection\)](#)

- [Update Firmware](#)

- [Install OS image](#)

[Firmware](#)

[AP/EC/ISH Firmware](#)

[Getting the Sources](#)

[Building the Firmware](#)

[Download the Released Firmware](#)

[Flashing Firmware](#)

[Flashing EC firmware](#)

[Flashing AP firmware](#)

[Reading GBB flags](#)

[Setting GBB flags](#)

[UFSC](#)

[Provision UFSC under AP console](#)

[Provision UFSC under EC console \(not yet available. b/463750635\)](#)

[Firmware Versions](#)

[Show logs for a specific app](#)

[Example for factory app](#)

[Factory App](#)

[Codebase](#)

[Prerequisites](#)

[Install Protobuf](#)

[Install Open JDK](#)

[Android Studio](#)

[Download Android Studio](#)

[Install Android Studio](#)

[Add ANDROID_SDK_ROOT to env](#)

[Running Android Studio](#)

[Tips: Add the program to the application launcher](#)

[Download SDK](#)

[Tips: Only Sync Factory repo](#)

[Import project in android studio](#)

[Generate APK sign keys from ARSP - Deprecated](#)

[Tips: For Virtual Device {Debug and test only}](#)

[Setup properties for build](#)

[Build and deploy apk](#)

[Build and Run without IDEs](#)

[Tips: Some Useful commands](#)

[Factory Framework](#)

[Test List](#)

[Framework Overview](#)

- [Layout & Key Files](#)
- [How it Works \(Execution Flow\)](#)
- [How to Add a New Test Action](#)
- [Summary of the Framework Logic](#)
- [Example](#)
- [Factory/MES Process](#)
 - [How it Works](#)
 - [Shopfloor](#)
 - [Software Overall Factory Process](#)
 - [Debug Tips](#)
 - [How to Expand ShopFloor](#)
- [Hardware Identification](#)
 - [Process Explanation](#)
- [Firmware](#)
- [Adb/Fastboot Debug Cable Setup](#)
 - [Install usbutils](#)
 - [Configure udev rules](#)
 - [Tips](#)
- [Git For LCFC](#)
 - [Internal Gitlab](#)
 - [Setup Internal Gitlab](#)
 - [Keep ARSP and internal Git sync](#)
 - [Push base code to rdgit - Admin Only](#)
 - [Use Local Branch \(Recommand\)](#)
 - [Admin](#)
 - [Developer](#)
 - [Upstream Sync {Sync base code from Google and Update}](#)
- [Common Tips](#)
 - [Disable LidSwitch On debian 13 with Lenovo thinkpad P16s](#)
 - [Edit the systemd logind configuration](#)
 - [Restart the systemd-logind service](#)
 - [Verify the changes](#)
 - [AG vs ARSP](#)
 - [Repo Diff](#)
 - [Adb Switch User](#)
 - [Check keyboard mapping](#)
 - [Mappings](#)
 - [Internal Proxy](#)
 - [Setup Internal Proxy via Web](#)

[Setup Internal Proxy via Terminal](#)

[BBR Activation for Linux](#)

[1. Configure BBR and FQ](#)

[2. Commit System Changes](#)

[3. Confirm Activation](#)

[4. Verify BBR Kernel Module Loading](#)

[Useful Links:](#)

[Open Questions](#)

[Document history](#)

Prepare

Environment

Linux Distribution (Optional)

Ubuntu

Download Ubuntu Desktop

<https://ubuntu.com/download/desktop>

Recommend using the latest LTS version for the best stability and support.

Install OS

Follow the on-screen instructions.

Debian (13 Trixie - stable channel)

- Debian is closer to Google's internal gLinux and has better stability than Ubuntu.
- Debian 因其出色的稳定性而闻名，Google 内部使用的 gLinux 便是基于Debian。
- For newer devices, maybe you can switch to using the testing channel.

Download Debian Live CD

<https://www.debian.org/CD/live/>

Recommend use Gnome Desktop

Install OS

You have the option to set a keyboard shortcut for Gnome Terminal, such as `Ctrl + Alt + T`.

Install Package

Shell

```
# Cleanup and update packages source
```

```
# 1. 删除旧的本地软件包清单
```

```
sudo rm -rf /var/lib/apt/lists/*
```

~~# 2. 强制重新下载最新的清单~~

~~sudo apt update~~

Get updates

sudo apt-get update

sudo apt-get upgrade

Install common package

sudo apt-get install git vim tmux tree gitk git-gui curl minicom wget

Install Software

Install Software: e.g: Chrome, Vscode, Antigravity etc.

Configure System Settings

Network, Theme, Keyboard Short, etc.

E.g: Theme: Setting -> Appearance -> Dark

Install Chinese Input

Shell

sudo vim /etc/environment

Append this

GTK_IM_MODULE=fcitx

QT_IM_MODULE=fcitx

XMODIFIERS=@im=fcitx

SDL_IM_MODULE=fcitx

GLFW_IM_MODULE=ibus

Switch to fcitx5

im-config -n fcitx5

Themes

sudo apt install fcitx5-material-color

配置 Fcitx5

重启电脑（或者注销重新登录），这样环境变量才会生效。

在应用菜单中找到 Fcitx 5 配置 (Fcitx 5 Configuration)。

在“当前输入法”列表中，确保 Pinyin（拼音）在其中。如果没有，在右侧搜索并添加。

Note: Ctrl + Space 切换输入法

```
# 手动创建自启动文件
## 如果 im-config 失效（在某些 Wayland 环境下会发生），直接在 GNOME 的自启动目录创建一个配置文件：
# 创建自启动文件夹（如果不存在）
mkdir -p ~/.config/autostart

# 将 Fcitx5 的启动快捷方式复制进去
cp /usr/share/applications/org.fcitx.Fcitx5.desktop ~/.config/autostart/
# 这样在你下次登录桌面时，GNOME 会强制运行 Fcitx5

reboot
```

Install VPN for Linux - Optional

Clash Party (Recommand)

<https://github.com/mihomo-party-org/clash-party/releases/tag/v1.8.8>

Open VPN Connect

```
Shell
sudo apt update
sudo apt install openvpn network-manager-openvpn
network-manager-openvpn-gnome
```

Then reboot the device and connect via network manager UI

Or use command line connect:

```
Shell
sudo openvpn --config ~/Downloads/qiao.ovpn
```

WireGuard

Import from network GUI

Terminal customization

Starship

Installation

Install the Starship cross-shell prompt using the package manager:

```
Shell
sudo apt install starship powerline fonts-powerline
```

Configuration

You can initialize a preset configuration to get started quickly. For example, to use the pastel-powerline preset:

```
Shell
starship preset pastel-powerline -o ~/.config/starship.toml
```

Apply Changes

To activate Starship, add the following initialization line to the end of your `~/.zshrc` or `~/.bashrc` file:

```
Shell
eval "$(starship init zsh)"
```

Note: For proper glyph and icon display, it is highly recommended to install and use a [Nerd Font](#) in your terminal emulator.

Zsh and Powerlevel10k (Optional, Not recommend)

Prerequisites

Before starting, ensure your package list is updated and you have the necessary tools (zsh, git, curl) installed.

```
Shell
sudo apt update
sudo apt install zsh git curl -y
```

Run the zsh installation script

```
Shell
sh -c "$(curl -fsSL
https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)
"

# Recommended Plugins (Optional)
# To add auto-suggestions and syntax highlighting, clone these into the
# plugins folder:

# Syntax Highlighting
git clone https://github.com/zsh-users/zsh-syntax-highlighting.git
${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zsh-syntax-highlighting

# Auto-suggestions
```

```
git clone https://github.com/zsh-users/zsh-autosuggestions.git
${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zsh-autosuggestions
```

```
# Then, edit your ~/.zshrc file and update the plugins line:
plugins=(git zsh-syntax-highlighting zsh-autosuggestions)
```

Set Zsh as default: During the installation, it will ask if you want to change your default shell to Zsh. Type **Y** and press Enter.

Note: If it doesn't ask, or if you skipped it, you can manually set it later by running:

```
Shell
chsh -s $(which zsh)

reboot
```

Install Powerline fronts

Using the Debian package manager (Recommended)

```
Shell
sudo apt-get install fonts-powerline
```

Update the font cache to ensure your system recognizes the new fonts:

```
Shell
fc-cache -vf
```

Install Powerlevel10k (p10k)

Now install the theme by cloning its repository into the Oh My Zsh custom themes directory.

```
Shell
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git
${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/themes/powerlevel10k
```

Setup theme to p10k

```
Shell
vim ~/.zshrc

# Set the theme Find the line that says ZSH_THEME="robbyrussell"

ZSH_THEME="powerlevel10k/powerlevel10k"
```

```
# Save and exit
```

Apply Changes

```
Shell  
source ~/.zshrc
```

Configure

Run the Configuration Wizard: If the wizard doesn't start automatically, type the following command:

```
Shell  
p10k configure
```

This will take you through a visual setup where you can choose your prompt style (Rainbow, Lean, etc.) and verify that icons are displaying correctly.

Git and Repo

[Git and Repo (Version Control)](<https://source.android.com/setup/develop>)

Git: is an open-source version-control system designed to handle very large projects that are distributed over multiple repositories.

- Git: <https://docs.github.com/en/get-started/start-your-journey/hello-world>

Repo: is a repository management tool that we built on top of Git. Repo unifies the many Git repositories when necessary.

- Repo: <https://gerrit.googlesource.com/git-repo>
<https://source.android.com/docs/setup/reference/repo>

Install Git

```
Shell  
sudo apt-get install git
```

Install Repo

Ubuntu

```
Shell  
sudo apt-get update  
sudo apt-get install repo
```

Debian

Ps: you can also skip this step and install `depot_tools`

<https://gerrit.googlesource.com/git-repo/+refs/heads/main/README.md>

```
Shell
sudo curl https://storage.googleapis.com/git-repo-downloads/repo -o
/usr/bin/repo

sudo chmod a+rx /usr/bin/repo

# Test
repo help
```

Configure git

```
Shell
git config --global user.name "User Name"
git config --global user.email "user-email@company.com"
```

Setup git credential

The `credential.helper` configuration tells Git how to store and retrieve credentials (like usernames and passwords or personal access tokens) for secure remote access.

For develop purpose:

```
Shell
git config --global credential.helper store
```

Docker

Official Reference

- Ubuntu <https://docs.docker.com/engine/install/ubuntu/>
- Debian: <https://docs.docker.com/engine/install/debian/>

Ubuntu

Please see official install page

Debian

Install using the apt repository

Set up Docker's apt repository

Shell

Add Docker's official GPG key:

```
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Add the repository to Apt sources:

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

```
sudo apt update
```

Install the Docker packages

Shell

```
sudo apt install docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

Verify that the installation is successful by running the hello-world image

Shell

```
sudo docker run hello-world
```

Setup permissions for docker and tty Users

Shell

```
sudo usermod -aG tty $USER && sudo usermod -aG docker $USER
```

Factory Server

Docker must be installed before installing the factory server.

Installation

Get deployment script

The deployment of all Chrome OS Factory Server components are managed by a `cros_docker.sh` script. To obtain that, you have three choices - just choose one of them:

- Download the latest version from the web. Make sure you have `curl` and `base64` then type following commands:

```
Shell
mkdir -p ~/factory-server
cd ~/factory-server
URL="https://chromium.googlesource.com/chromiumos/platform/factory/+/main/setup/cros_docker.sh?format=TEXT"
curl -L "${URL}" | base64 --decode >cros_docker.sh
bash cros_docker.sh update # Self-test and change file permission.
```

- Or, check out from factory software repository:

```
Shell
mkdir -p ~/chromiumos/src/platform
cd ~/chromiumos/src/platform
git clone https://chromium.googlesource.com/chromiumos/platform/factory
cd factory/setup
# Now you have ./cros_docker.sh.
```

Download and install factory server

Run following command to download the Docker image on a machine which has access to internet:

```
Shell
bash cros_docker.sh pull
```

If your target server machine does not have a public network, copy `cros_docker.sh` and files listed on the screen to the target computer.

On the target server machine, run following command to load Docker images:

```
Shell
bash cros_docker.sh install
```

Run the following command to start the front end Dome. Note: When we uprev the UMPIRE_VERSION, Umpire instances will be updated automatically after we run the following command for the first time.

```
Shell
bash cros_docker.sh run
```

Update deployment script and server images

The version of the server image is tracked inside deployment script cros_docker.sh. To update deployment script and server images to latest version, do:

```
Shell
bash cros_docker.sh update
```

Then repeat the steps in the previous section to update Docker images.

Build Factory Server

Build the dev version factory server

```
Shell
bash cros_docker.sh build
```

Run the dev version factory server

```
Shell
bash cros_docker.sh run
```

Debug

```
Shell
# For Testing Version (if you can't see some new feature)
setup/cros_docker.sh build && ./setup/cros_docker.sh run
```

Access

```
Shell
http://localhost:8000

# user name: admin
# user password: test0000
```

Fastboot

https://docs.google.com/document/d/1tflz9kl_selRMhZGZrEcwEvT9-1ZnWMTRgZkQDS5fGU/edit?tab=t.0

Important commands

The following commands are integral to the automated imaging workflow. These examples are provided for users who wish to execute the process manually using raw commands.

Flash boot partition

```
Shell
fastboot -s tcp:${DUT_IP}:5554 flash boot
```

Flash all partitions in \$ANDROID_PRODUCT_OUT defined in fastboot-info.txt

```
Shell
fastboot -s tcp:${DUT_IP}:5554 flashall
```

Flash MBR-GPT binary

```
Shell
fastboot -s tcp:${DUT_IP}:5554 flash raw-sector:0 mbr-gpt.bin
```

Get key variables

```
Shell
fastboot -s tcp:${DUT_IP}:5554 getvar all
```

Get FW variant or board name (depending on fastboot mode)

```
Shell
fastboot -s tcp:${DUT_IP}:5554 getvar product
```

(FW fastboot) -> dochi

(Userspace fastboot) -> brya

Reboot to bootloader mode (has to be executed in userspace fastboot)

```
Shell
fastboot -s tcp:${DUT_IP}:5554 reboot bootloader
```

Reboot to userspace fastboot mode (has to be executed in FW fastboot)

```
Shell
fastboot -s tcp:${DUT_IP}:5554 reboot fastboot
```

Reboot device

```
Shell
fastboot -s tcp:${DUT_IP}:5554 reboot
```

Set active slot as a

```
Shell
fastboot -s tcp:${DUT_IP}:5554 set_active a
```

Check current FW mode

```
Shell
fastboot -s tcp:${DUT_IP}:5554 getvar is-userspace
```

(FW fastboot) -> no

(Userspace fastboot) -> yes

Apart from disk provisioning

In addition to standard flash commands, OEMs have the flexibility to implement specialized augmented commands to satisfy specific business requirements.

Representative OEM commands supported in AL include:

Change selinux

```
Shell
oem bootconfig set "androidboot.selinux=permissive"
```

Enable AP console

Shell

```
oem cmdline set "console=ttyS0,115200 printk.devkmsg=on initcall_debug  
DEBUG=2"
```

Set successful boot flag

Shell

```
oem set-successful:vbmeta_b:1
```

Get kernel boot priorities

Shell

```
oem get-kernels
```

```
(bootloader) a:boot_a:prio=2
```

```
(bootloader) b:boot_b:prio=0
```

Fastboot with DOME Server

1. Switch the active toggle and press **DEPLOY**, then fastboot server would be running in the background. To stop it, deactivate the toggle and **deploy again**.

sole

Docker Image: 2
Hash: *0774af00

Dashboard

☒ Enable Umpire

Info

port: 9090

Services

fastbootService

☐ active

dutIpAdrrs

dutip

192.168.1.0/24

×

+

scanInterval

10

boardName

brya

modelName

redrix

idleTimeout

60

DISCARD CHANGES

DEPLOY

Fastboot with Fastboot Orchestrator

The fastboot orchestrator is a standalone CLI designed to execute the image flashing procedure.

More specifically, the tool follows these defined steps to flash a device:

- Utilizes IP scanning to locate a device in FW fastboot mode.
- Once identified, proceeds to flash the GPT, followed by the essential boot partitions.
- Reboots the device into fastbootd (userspace fastboot), leveraging its superior networking support compared to firmware.
- Continues with the flashing of remaining partitions.
- Triggers a final reboot once all flashing operations are complete.

To execute the fastboot orchestrator CLI directly, individual partitions must be utilized rather than a unified `preflash.img`. This necessitates the following procedures:

- Extract discrete partitions (`boot`, `init_boot`, `vendor_boot`, `pvmfw`, `vbmeta`, `super`) from the `preflash.img`.
- Generate a `fastboot-info.txt` file with the designated configuration:

None

```
# fastboot-info
version 1
flash boot
flash init_boot
flash pvmfw
flash vendor_boot
flash --apply-vbmeta vbmeta
flash super
erase metadata
```

- Consolidate all partitions, the `GPT.bin`, and `fastboot-info.txt` into a single directory, then provide this path to the orchestrator as the `src_image_dir`.

How to debug

General

Inspect `/cros_docker/umpire/[project_name]/log/fastboot_service.log` for relevant logging output.

How do I know the progress of flashing?

Currently, Fastboot and fastbootd do not provide real-time progress indicators on the UI. To monitor the specific stage of the DUT, consult the `fastboot_service.log` within `/cros_docker/umpire/[project_name]/log/`.

Factory Server Development

Network Configuration for Factory Server

To enable communication between the Factory Server and the Device Under Test (DUT), the host machine must be configured to share its network connection via DHCP.

Recommended Method: Using NetworkManager (shared)

This is the modern and simplified approach for setting up a DHCP server on your host using standard Linux network tools.

1. **Identify the Network Interface:** Connect your USB-to-Ethernet adapter and find its name (usually starting with `enx`):

```
Shell
ip a
```

2. **Create the Shared Connection:** Use `nmcli` to create a new profile. Replace `[INTERFACE_NAME]` with your actual device name:

```
Shell
nmcli con add type ethernet ifname [INTERFACE_NAME] con-name
FastbootShared ipv4.method shared ipv6.method ignore
```

3. **Verify Activation:** Check that the connection is active and has assigned a gateway IP (typically 10.42.x.1):

```
Shell
nmcli connection show
```

FYI, for the bash or the zsh env (`.zshrc`)

```
Shell
# NetworkManager DHCP 共享, 开机本机DHCP
# Fastboot 网络共享
fs-on() {
    local dev=$(ip -o link show | grep 'enx' | tail -n1 | awk -F': ' {
    print $2})
    [ -z "$dev" ] && echo "❌ 未发现 USB 网卡" && return 1
```

```

# 删旧 -> 建新 -> 启动
nmcli con delete FastbootShared 2>/dev/null
nmcli con add type ethernet ifname "$dev" con-name FastbootShared
ipv4.method shared ipv6.method ignore >/dev/null
nmcli con up FastbootShared && echo "✅ FastbootShared 已就绪 ($dev)"
}

fs-off() {
    nmcli con delete FastbootShared 2>/dev/null && echo "🔴 FastbootShared
已清理"
}

```

Legacy Netboot/DHCP Setup (Deprecated)

Note: These methods are preserved for reference only. Aluminum OS generally does not require these manual dnsmasq or TFTP configurations.

Manual dnsmasq Setup (Deprecated)

To install and configure dnsmasq manually for static interface environments:

```

Shell
# Install the service
sudo apt-get install dnsmasq

# Manual execution for debugging
sudo dnsmasq -d -q -i [INTERFACE] --dhcp-range=10.0.0.50,10.0.0.100,12h
--dhcp-option=3,10.0.0.10 --log-dhcp

```

~~TFTP Configuration (Deprecated)~~

```

Shell
# Create directory and set permissions
sudo mkdir -p /var/tftp
sudo chown -R dnsmasq:nogroup /var/tftp

# Configuration lines for dnsmasq.conf
enable tftp
tftp root=/var/tftp

```

```
dhcpcd-option=option:tftp-server,10.0.0.10
```

Setup TFTP server and DHCP server with dnsmasq

Install dnsmasq

```
Shell  
sudo apt-get install dnsmasq
```

Check network interface

```
Shell  
ip link  
# or  
ip a  
# or  
ip addr show
```

Configure network interface

```
Shell  
sudo vim /etc/network/interfaces
```

Append

```
Shell  
# Local settings for Netboot/Fastboot  
allow-hotplug enp195s0f0  
  
iface enp195s0f0 inet static  
—— address 10.0.0.10  
—— netmask 255.255.255.0
```

Apply settings

```
Shell  
sudo ip addr flush dev enp195s0f0  
sudo ifdown enp195s0f0  
sudo ifup enp195s0f0  
  
# Verify  
ip addr show enp195s0f0
```

```
# Need show: inet 10.0.0.10/24
```

Decide a folder to put TFTP files, for example, /var/tftp

Shell

```
sudo mkdir /var/tftp  
sudo chown "${USER}" /var/tftp  
# mkdir -p "/var/tftp/${MODEL}"  
echo "TFTP Test Successful!" | sudo tee /var/tftp/test.txt
```

Setup tftp Permission

Shell

```
# set permission (make sure dnsmasq process can read/write)  
sudo chown R nobody:nogroup /var/tftp  
sudo chmod R 755 /var/tftp
```

Configure dnsmasq (DHCP + TFTP)

Create dnsmasq config

Shell

```
sudo vim /etc/dnsmasq.d/fastboot.conf
```

Add content

Shell

```
huanhuan1 ~ /aluminium/ruby ♥ 14:14 cat /etc/dnsmasq.d/fastboot.conf  
port=0
```

```
# 网卡
```

```
interface=enx0826ae3acde7  
interface=enx0826ae3acdd3
```

```
bind-dynamic
```

```
# DHCP (关键: 绑定接口)
```

```
dhcp-range=enx0826ae3acde7, 10.0.0.50, 10.0.0.100, 255.255.255.0, 12h  
dhcp-range=enx0826ae3acdd3, 10.0.1.50, 10.0.1.100, 255.255.255.0, 12h
```

```
# 网关
```

```
dhcp-option=3, 10.0.0.10  
dhcp-option=3, 10.0.1.10
```

```
# DNS  
dhcp-option=6,8.8.8.8  
  
# lease  
dhcp-leasefile=/tmp/dnsmasq.leases  
  
log-dhcp
```

Check dnsmasq config file

```
Shell  
sudo dnsmasq -t -C /etc/dnsmasq.d/fastboot.conf
```

Check and start dnsmasq

```
Shell  
# restart dnsmasq  
sudo systemctl restart dnsmasq  
  
# Check status  
sudo systemctl status dnsmasq  
# Please make sure "Active: active (running)" is showing  
  
# Check Log  
sudo journalctl -u dnsmasq -f -n 100
```

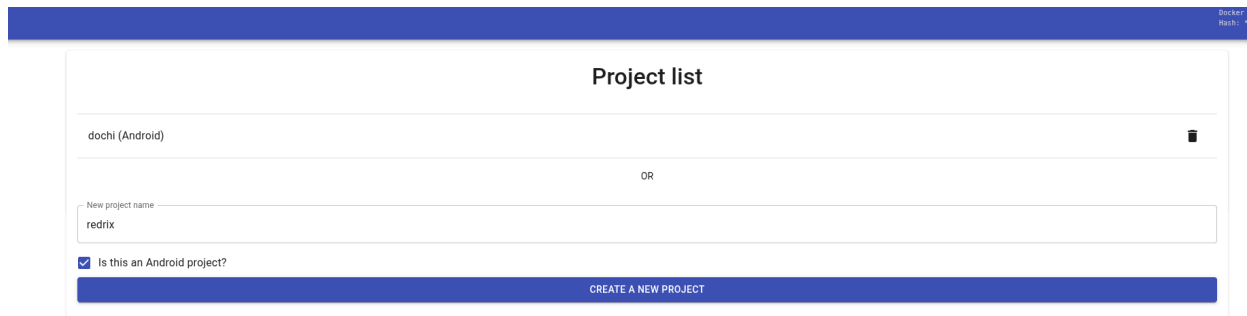
Manually run dnsmasq {Debug}

```
Shell  
# First, need stop system service  
sudo systemctl stop dnsmasq  
  
# Start and monitor  
sudo dnsmasq -d -q -c conf file=/etc/dnsmasq.d/fastboot.conf
```

Setup DOME server for fastboot

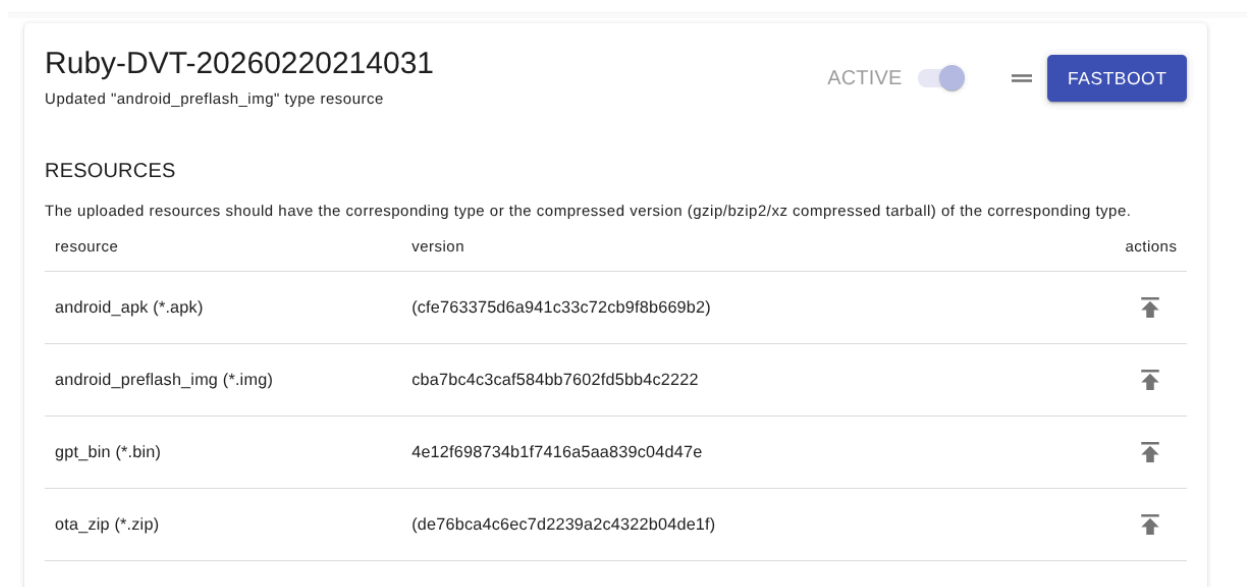
Fastboot with DOME / UMPIRE server

On the browser side, connect to the DOME server (by default it's <http://localhost:8000>). In the landing page, create a new project with its arbitrary name, and toggle the **Is this an Android project**



The screenshot shows the 'Project list' interface. At the top, there's a header 'Project list'. Below it, a form contains a text input for 'New project name' with the value 'redrix'. A checkbox labeled 'Is this an Android project?' is checked. A blue button at the bottom says 'CREATE A NEW PROJECT'. In the top right corner, there's a small status bar showing 'Docker: Ready'.

Once the project is created, left-click into it and enable the Umpire service. Then, go to the "Bundles" page to upload the factory preflash image and the GPT.bin, both prepared during the prerequisites phase.



The screenshot shows the 'Bundles' page for a project named 'Ruby-DVT-20260220214031'. The page has a header with the project name, a status 'Updated "android_preflash_img" type resource', an 'ACTIVE' toggle switch, and a blue 'FASTBOOT' button. Below the header is a section titled 'RESOURCES' with a description: 'The uploaded resources should have the corresponding type or the compressed version (gzip/bzip2/xz compressed tarball) of the corresponding type.' A table lists the resources:

resource	version	actions
android_apk (*.apk)	(cfe763375d6a941c33c72cb9f8b669b2)	⬆
android_preflash_img (*.img)	cba7bc4c3caf584bb7602fd5bb4c2222	⬆
gpt_bin (*.bin)	4e12f698734b1f7416a5aa839c04d47e	⬆
ota_zip (*.zip)	(de76bca4c6ec7d2239a2c4322b04de1f)	⬆

After the resource is uploaded, click the "FASTBOOT" button on the top right corner to designate this bundle as the proper bundle for fastboot imaging.

- To utilize a specific bundle for flashing with Fastboot, you must click the "FASTBOOT" button. Simply toggling the "ACTIVE" status of a bundle will not change the bundle that fastboot uses.

On the “**dashboard**” page, locate the “fastboot service” dropdown. Here, configure the server with the following arguments

- dutIpAdrrs: The desired IP addresses to scan for fastboot. This field accepts either a single IP address or a CIDR range. To add multiple IP addresses, click the “+” button in the top right corner.
- ScanInterval: The interval for scanning the devices in the IP range. The default is 10 seconds.
- BoardName: The board name of the DUT, which is used by userspace fastboot to check if the DUT is the target board that we want to flash.
- ModelName: The model name of the DUT, which is used by firmware fastboot to check if the DUT is the target board that we want to flash.
- IdleTimeout: The maximum idle duration for a single flashboot operation. The default is 60 seconds.

Switch the active toggle and press **DEPLOY**, then fastboot server would be running in the background. To stop it, deactivate the toggle and **deploy again**.

The screenshot shows a terminal window with a dark blue header bar. On the left, the word "console" is partially visible. On the right, it says "Docker Image: 2" and "Hash: *8774af08". The main content area is titled "Dashboard" and contains the following elements:

- A toggle switch labeled "Enable Umpire" which is currently turned on (blue).
- An "Info" section showing "port: 9090".
- A "Services" section with a dropdown menu currently showing "fastbootService".
- Below the dropdown, there is a toggle switch labeled "active" which is currently turned off (grey).
- A "dutIpAdrrs" field with a list of IP addresses. One address, "192.168.1.0/24", is visible in a box, and a "+" button is to its right.
- A "scanInterval" field with the value "10".
- A "boardName" field with the value "brya".
- A "modelName" field with the value "redrix".
- An "idleTimeout" field with the value "60".
- At the bottom, there are two buttons: "DISCARD CHANGES" and "DEPLOY".

Debug Tips

Enter umpire shell

```
Shell
bash cros_docker.sh umpire shell
```

Aluminium OS

Context

Objective

Become familiar with the various components of AL and skillfully apply them.

Background

This document outlines the learning process for transitioning from the ChromeOS factory to the AL architecture.

Prerequisites

This section describes the system requirements and one time system package setup. For latest instructions, please see this page: [AOSP setup instructions](#)

System Requirements

Please ensure that your system meets following hardware requirements:

- A 64 bit x86 system (>16 threads)
- At least 2TB of the free disk space
- A minimum of 64 GB RAM
- Ubuntu 22.04 or later (required to build Android version >= 11)
- Debian 13 trixie (required to build Android version >= 11) - Recommended

Install Required Packages

```
Shell
sudo apt install git-core gnupg flex bison build-essential zip curl
zlib1g-dev libc6-dev-i386 x11proto-core-dev libx11-dev lib32z1-dev
libgl1-mesa-dev libxml2-utils xsltproc unzip fontconfig lz4
device-tree-compiler
```

Account Registration

Register account with ARSP(<https://arsp-review.google.com/dashboard/self>) and have the access agreement in place for the provision. Once provision is confirmed please proceed next.

Generate git cookies

For the first time, users will need to update credentials for arsp repositories.

Please follow the steps below to generate a .gitcookies file to access arsp and sync the code.

1. Navigate to <https://www.google.com/new-password>
2. Login to the above page using corp-partner account (*corp-partner.google.com)
3. It will open the window with http credentials (*this is the sample snippet, please copy paste the snippet shown on your browser*)

```
eval 'set +o history' 2>/dev/null || setopt HIST_IGNORE_SPACE 2>/dev/null
touch ~/.gitcookies
chmod 0600 ~/.gitcookies

git config --global http.cookiefile ~/.gitcookies

tr , \\t <<\__END__ >>~/.gitcookies
partner-android.google.com,FALSE,/,TRUE,2147483647,o,git-maulikvaghela.crotest.corp-
partner-android-review.google.com,FALSE,/,TRUE,2147483647,o,git-maulikvaghela.crotest
__END__
eval 'set -o history' 2>/dev/null || unsetopt HIST_IGNORE_SPACE 2>/dev/null
```

4. Copy entire text and execute it on the linux setup.
Please note that this can be executed from any directory.

Get the Source Code

Once developers have access (or added to the correct PD group), the below command needs to be executed to initialize the repository to download the source code.

```
Shell
cd ~
mkdir -p aluminium/ruby
cd aluminium/ruby

# 0212 Test: this step will cause sync & build fail
REPO_ALLOW_SHALLOW=0 repo init -c -u
"https://arsp.google.com/platform/manifest" --use-superproject
--partial-clone --partial-clone-exclude=platform/frameworks/base
--clone-filter=blob:limit=10M -b device/ruby/main
```

Fully Sync, Need Storage > 2TB

```
REPO_ALLOW_SHALLOW=0 repo init -c -u  
https://arsp.googleusercontent.com/platform/manifest --use-superproject -b  
device/ruby/main
```

For branch

```
REPO_ALLOW_SHALLOW=0 repo init -c -u  
https://arsp.googleusercontent.com/platform/manifest --use-superproject -b  
device/ruby/main -m manifest_device.xml  
REPO_ALLOW_SHALLOW=0 repo init -c  
https://arsp.googleusercontent.com/platform/manifest --use-superproject -m  
commit-hash
```

Sync

```
$ repo sync -c -j $(nproc)
```

Update (Will only sync the have changed repos)

```
$ repo sync -c -j $(nproc) --optimized-fetch
```

Set up option for replacing prebuilts

```
$ ln -sf vendor/google/desktop/dev/kernel/replace_prebuilts.py .
```

Branch Build - ARSP 20260417 update

On or post **arsp-main** build **15054525**: First look into the manifest repository

[https://arsp.googleusercontent.com/platform/manifest/device/\\${BOARD}](https://arsp.googleusercontent.com/platform/manifest/device/${BOARD}) to find the corresponding build ID, or the latest build ID as it is the one most close to the tip of the tree, in the form of “... Busytown Build ID XXXXXXXX” where XXXXXXXX is the build number. For the given Build ID this indicates the availability of the versioned manifest as the manifest may not be published for all builds. Then initialize workspace with,

Shell

```
$ export BOARD=<BOARD> ### e.g. fatcat -- for PTL RVP
```

```
$ export PLATFORM=<PLATFORM> ### e.g. fatcat SoC platform
```

```
$ mkdir -p <WORK_DIR>/alos/${BOARD} && cd <WORK_DIR>/alos/${BOARD}
```

```
### Checkout ARSP against commit hash of the manifest (XXXXXXX=Build ID)
```

```
$ REPO_ALLOW_SHALLOW=0 repo init -c -u
https://arsp.googleusercontent.com/platform/manifest/device/${BOARD}
--partial-clone --clone-filter=blob:limit=10M -b refs/buildid/XXXXXXX
```

After repository initialization, either from 1, or 2, sync with the following,

```
Shell
### Sync'ing codes and create default local branch "def"
$ repo sync -c -j $(nproc) --optimized-fetch && repo start def --all

### Set up option for replacing prebuilts
$ ln -sf vendor/google/desktop/dev/kernel/replace_prebuilts.py .
```

Images

Build Images

```
Shell
# Sync the repositories
repo sync -c -j $(nproc)

# Set up the build environment source
source build/envsetup.sh

bash +x
vendor/google_shared/packages/desktop/ApkTempPrebuilts/arsp-apks-prebuilt-
recipe.sh

# Configure the build for your device
lunch ruby-trunk_staging-userdebug
# lunch ruby-arsp_trunk_staging-userdebug

# Start new clean build
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false m installclean

# Start the build
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false m
```

```
# Build images
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false m pack-image
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false m dist

# Tips
m dist                # generates combined image (Release & Recovery)
m pack-image          # generate only Android desktop Image
m pack-recovery-image # generate Android desktop recovery Images
```

Tips: Reset git commit head

```
Shell
# If lost some package, please enter path and run
git reset --hard HEAD
# then
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false m pack-image
```

Tips: Install the Google Cloud CLI

```
Shell
sudo apt-get update
sudo apt-get install ca-certificates gnupg curl

curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo gpg
--dearmor -o /usr/share/keyrings/cloud.google.gpg

echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg]
https://packages.cloud.google.com/apt cloud-sdk main" | sudo tee -a
/etc/apt/sources.list.d/google-cloud-sdk.list

sudo apt-get update && sudo apt-get install google-cloud-cli

gcloud auth login
# Please use partner account login
```

Building Kernel

Initialize the build environment and configure the target device.

```
Shell
cd <WORK_DIR>/alos/${BOARD}
```

```
export BUILD=userdebug # or eng
source build/envsetup.sh
```

Note: Use the following workaround to incorporate pre-built APKs within the ARSP environment.

```
Shell
bash +x
./vendor/google_shared/packages/desktop/ApkTempPrebuilts/arsp-apks-prebuilt-recipe.sh
```

Select the appropriate lunch target (e.g., *arsp_trunk_staging* for *arsp-main* or *cl2b* for the *26Q2* branch).

```
Shell
lunch ${BOARD}-arsp_trunk_staging-${BUILD}
```

Using Kernel Prebuilts for Android image

```
Shell
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false m installclean
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false m pack-image
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false m dist
```

Following a successful build, the resulting artifacts—including *android-desktop_image.bin*—are located in the *out/target/product/\${BOARD}/* directory.

Using Locally Built Kernel as Replacement (Optional)

Procedures for cloning kernel repositories and building from source.

```
Shell
mkdir <WORK_DIR>/alos/kernel && cd <WORK_DIR>/alos/kernel
```

Fetch from ARSP

Shell

```
repo init -u https://arsp.googleusercontent.com/kernel/manifest -b  
android17-6.18-desktop-intel # or SoC-specific branch
```

Shell

```
repo sync -c -j $(nproc) && repo start def --all
```

Shell

```
cd <WORK_DIR>/alos/${BOARD}
```

Shell

```
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false ./replace_prebuilts.py  
--build-kernel --build-android --external <WORK_DIR>/alos/kernel
```

Using Locally Built Kernel as Replacement with version'd manifest (Optional)

To utilize a specific ARSP build, first execute `uname -a` to identify the kernel pre-built version (formatted as `abXXXXXXXX`). Locate the corresponding build on the [Android CI](#) and download the versioned manifest (`manifest_XXXXXXXX.xml`). This file is used during repository initialization via the `-m` flag.

Initialize kernel repositories using the specific manifest:

Shell

```
mkdir <WORK_DIR>/alos/kernel && cd <WORK_DIR>/alos/kernel  
mkdir -p .repo/manifests  
cp ~/Downloads/manifest_XXXXXXXX.xml .repo/manifests
```

Fetch from ARSP

Shell

```
repo init -u https://arsp.googleusercontent.com/kernel/manifest -b  
android17-6.18-desktop-intel -m manifest_XXXXXXX.xml
```

Shell

```
repo sync -c -j $(nproc) && repo start def --all
```

Shell

```
cd <WORK_DIR>/alos/${BOARD}
```

Build and replace kernel

Shell

```
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false ./replace_prebuilts.py  
--build-kernel --build-android --external <WORK_DIR>/alos/kernel
```

Make Pre-flash Image

To flash the device with fastboot, the following two payloads are needed.

- the preflash.img, an single image with the defined partition layout
- the GPT.bin, an GUID partition table with MBR header

Both of them can be generated with image_tool.

```
$ARSP_ROOT/vendor/google_shared/packages/desktop/Factory/factory  
/image_tool.py
```

To generate preflash.img and GPT.bin, prepared the following contents and pass them to image_tool

Shell

```
image_tool.py \  
-d /path/to/userdebug_image.bin \  
-o /path/to/output/preflash.img \  
-g /path/to/output/mbr-gpt.bin \  
--user_data_size USER_DATA_SIZE_IN_GiB \  

```

```
--gpt_sector_size SECTOR_SIZE_IN_BYTE
--factory_app /path/to/factory_app.apk \
--ota_package /path/to/ota_package.zip
```

Example:

```
vendor/google_shared/packages/desktop/Factory/factory/image_tool.py
--userdebug_image ~/Downloads/android-desktop_image.bin --factory_app
~/workspace/rdgit/Factory/factory/app/build/outputs/apk/debug/app-debug.ap
k --output_gpt ~/Downloads/0220-gpt.bin --ota_package
~/Downloads/ruby-ota-14905995.zip --output
~/Downloads/ruby_preflash_0220.bin
```

Example2:

```
vendor/google_shared/packages/desktop/Factory/factory/image_tool.py
--userdebug_image out/target/product/ruby/android-desktop_image.bin
--factory_app
~/aluminium/ruby/vendor/google_shared/packages/desktop/Factory/factory/app
/build/outputs/apk/release/app-release.apk --output_gpt
~/Downloads/ruby-GPT-20260306-arsp_v1.bin --ota_package
out/target/product/ruby/ruby-ota.zip --output
~/Downloads/ruby_preflash_20260306_arsp_v1.bin
```

Example3:

```
~/aluminium/ruby/vendor/google_shared/packages/desktop/Factory/factory/ima
ge_tool.py --userdebug_image android-desktop_image.bin --factory_app
app-release-20260306-for-TPE-runin48H_v1.apk --output_gpt GPT-14941224.bin
--output ruby_preflash_20260308_ZL1B.260225.001_14941224.bin
```

Tips: Install lpadd lpdump lpunpack

Shell

Error message

FileNotFoundError: ['lpadd', 'lpdump', 'lpunpack'] is missing. If you have android source, run "m lpadd lpdump lpunpack" to build it.

Set up the build environment source

```
source build/envsetup.sh
```

Configure the build for your device

```
lunch ruby-trunk_staging-userdebug
```

Only build three modules

```
# SOONG_ALLOW_MISSING_DEPENDENCIES=true SOONG_RUN_CIPD_PROXY_SERVER=false  
USE_RBE=false m lpadd lpdump lpunpack
```

```
SOONG_RUN_CIPD_PROXY_SERVER=false USE_RBE=false m lpadd lpdump lpunpack
```

Tips: Debug file mtime error

Shell

```
# 进入 ruby 项目根目录
```

```
cd ~/aluminium/ruby
```

```
# 1. 强行重置所有受版本控制的文件时间（最快且覆盖面广）
```

```
repo forall -c "git ls-files -z | xargs -0 touch"
```

```
# 2. 补漏：重置那些不在 git 管理下但可能在未来的 bp 和工具文件
```

```
find . -newermt "now" -exec touch -h {} +
```

```
# 3. 彻底删除 Soong 的构建快照（这一步由于时间戳混乱是必须的）
```

```
rm -rf out/soong/.bootstrap out/soong/build.ruby.ninja*
```

Flashing Aluminium

Disable WP (write-protection)

Please follow device specific instructions to disable write-protection if in need.

Update Firmware

Use device specific instructions for updating AP and EC firmware.

Install OS image

Load Aluminium image to USB flash drive and use it to boot the DUT (assume `/dev/sda` is the flash drive),

Shell

```
sudo dd
```

```
if=<WORK_DIR>/alos/${BOARD}/out/target/product/${BOARD}/android-desktop_image.bin of=/dev/sda bs=8M conv=fsync,nocreat status=progress
```

The example above used the locally built desktop image. However, it is also possible to download the images from the builders and make use of those as well.

From the ARSP build, visit

<https://ci.android.com/builds/branches/arsp-main/grid?legacy=1> then find the build targets corresponding to the given `${BOARD}`.

After booting from the USB flash drive (typically via dev mode with “Boot from external disk” option), install the image to the internal disk from the serial console, or adb shell. Serial console is by default enabled on Eng build image:

```
Shell
su
# echo y | android-desktop-install
```

Firmware

AP/EC/ISH Firmware

Getting the Sources

```
Shell
mkdir -p <WORK_DIR>/cros/${BOARD} && cd <WORK_DIR>/cros/${BOARD}

repo init -u https://chromium.googlesource.com/chromiumos/manifest.git
--repo-url https://chromium.googlesource.com/external/repo.git

repo sync -j4 -c

./src/config/setup_project.sh --checkout=$(pwd) -program ${PLATFORM}
-project ${BOARD}

repo sync -j4 -c

repo start def --all    ### "def" -- local branch name
```

Building the Firmware

```
Shell
cros_sdk --enter

setup_board --board=${PLATFORM} --profile=${BOARD} --force
```

```
cros build-packages --board=${PLATFORM} --profile=${BOARD} --autosetgov
--accept-licenses Google-TOS virtual/chromeos-firmware
chromeos-zephyr-ish chromeos-bsp-${PLATFORM}-private
```

AP/EC/ISH firmware artifacts should be found under `/build/${PLATFORM}/firmware/` directory. For loading ISH firmware the firmware file needs to be renamed to `ish_pt1_62b0f067_6c687233.bin` (per Intel's specifications).

Download the Released Firmware

Instead of relying on locally built firmware, Release firmware can also be downloaded from DLM.

Flashing Firmware

Via servo connection,

Flashing EC firmware

```
Shell
flash_ec --board=fatcat_npcx9m7f --zephyr --image=ec.bin --chip=npcox_uut
```

Flashing AP firmware

```
Shell
futility update --servo --image=image-ruby.serial.bin
```

Reading GBB flags

```
Shell
futility gbb --servo --get --flags
```

Setting GBB flags

```
Shell
futility gbb --servo --set --flags=<flags>
```

UFSC

Intercepting EVT build is the introduction of UFSC. For the proto board, additional CBI provision is needed to be compatible. This is involved with the additional provisioning of CBI tag 29 (UFSC field),

Provision UFSC under AP console

```
Shell  
ectool cbi set ufsc 49108046000800000000000000000000
```

Provision UFSC under EC console (not yet available, b/463750635)

Please refer to the references for more details,

https://chromium.googlesource.com/chromiumos/config/+refs/heads/main/util/ufsc/ufs_c_developer_guide.md

https://source.chromium.org/chromiumos/chromiumos/codesearch/+/-/main:src/platform/ec/docs/zephyr/zephyr_ufsc.md

Firmware Versions

To verify the currently installed firmware version on the DUT, execute the following commands:

```
Shell
adb shell crossystem
```

```
Shell
adb shell ectool version
```

Show logs for a specific app

Replace <package_name> with the target package

```
adb logcat --pid=$(adb shell pidof <package_name>) -v color
```

Example for factory app

Monitor logs for the factory application

Shell

```
adb logcat --pid=$(adb shell pidof com.google.android.factory.factory) -v color
```

Factory App

This factory software is used to run factory tests, and it can be viewed as the factory toolkit in ChromeOS.

Google shifted the factory process from DSU (Dynamic System Updates) to OTA (Over-The-Air).

DSU: {userdebug + user + vendor_boot-debug + factory app} -> {wiping unnecessary materials -> user image}

OTA: {userdebug + factory app -> OTA to user image (ota_package.zip from DOME)}

Codebase

Shell

Factory App:

```
arsp/$BOARD/vendor/google_shared/packages/desktop/Factory/factory
```

Prerequisites

Install Protobuf

Follow Protobuf.dev web site to Install protocol buffer compiler

Shell

```
sudo apt install protobuf-compiler
sudo apt install python3-protobuf
sudo apt install gdisk
```

```
protoc --version
```

```
# Ensure compiler version is 3+, e.g:
# libprotoc 3.21.12
```

Install Open JDK

Shell

```
sudo apt update
```

```
# The LTS version recommends 17 or 21, can select one for project
```

```
sudo apt install openjdk-17-jdk  
sudo apt-get install default-jdk
```

Android Studio

Download Android Studio

Visit the official Android developer website to download the latest Linux tar.gz archive.

Official download page: <https://developer.android.com/studio>

The file name you downloaded will be similar to [android-studio-202X.X.X-linux.tar.gz](#)

Install Android Studio

Decompress and move to the/opt directory

Installing software to the/opt directory is a standard method for storing third-party software in Linux systems for easy management.

```
Shell  
#Enter the download directory  
cd ~/Downloads  
  
# Decompress and move to /opt (sudo is required)  
sudo tar -xzvf android-studio-*-linux.tar.gz -C /opt  
  
# Change permissions (make the current user have writable permissions)  
sudo chown -R $USER:$USER /opt/android-studio
```

Add ANDROID_SDK_ROOT to env

```
Shell  
vim ~/.bashrc or ~/.zshrc  
  
# Append this  
  
# >>> Android SDK environment variables <<<  
export ANDROID_HOME=$HOME/Android/Sdk  
export ANDROID_SDK_ROOT=$HOME/Android/Sdk  
  
# Add Android Tools to PATH  
export PATH=$PATH:$ANDROID_HOME/platform-tools  
export PATH=$PATH:$ANDROID_HOME/emulator  
export PATH=$PATH:$ANDROID_HOME/cmdline-tools/latest/bin
```

```
export PATH=$PATH:$ANDROID_HOME/tools
export PATH=$PATH:$ANDROID_HOME/tools/bin

# Save and apply
# source ~/.bashrc
source ~/.zshrc
```

Running Android Studio

```
Shell
bash /opt/android-studio/bin/studio.sh
```

Tips: Add the program to the application launcher

(Tools > Create Desktop Entry...) is the correct way to add it to your native launcher bar.

After you do that, you can:

- Open your application launcher.
- Find "Android Studio".
- Right-click it and select "Add to Favorites" or "Pin to Dash/Dock" to keep it on your bar.

After completing the preceding steps, you can open it from Apps. By default, you can use Standard to install it.

Download SDK

1. Click More Actions -> SDK Manager
2. Download "Android SDK Platform" and "Sources for Android" with API level upper 35
3. Choose the path to download SDK. By default it is ~/Android/Sdk.
4. Click Apply

Note: SKD Platforms can select multiple API levels at the same time, the factory is currently using 35, remember to check the SDK Command line tool for SDK tools.

Tips: Only Sync Factory repo

Note: Just for testing, factory repo now depends-on pre-built packages, this method is deprecated.

```
Shell
git clone
https://arsp.googleusercontent.com/platform/vendor/google_shared/packages/desktop/Factory
```

Import project in android studio

Import the project from

```
`$ARSP_ROOT/vendor/google_shared/packages/desktop/Factory/`
```

After importing, wait for gradle sync to complete. If you encounter gradle download network problems, please change the network and try again.

Generate APK sign keys from ARSP - Deprecated

By default, Soong signs platform applications with the key located in \$ARSP_ROOT/build/target/product/security/. The following commands generate the keystore for Gradle as \$PATH_OF_OUTPUT_KEYSTORE_FILE:

Shell

```
# Generate platform.key  
openssl pkcs8 -inform DER -nocrypt -in  
~/aluminium/ruby/build/target/product/security/platform.pk8 -out  
platform.key  
  
# Generate platform.p12  
openssl pkcs12 -export -in  
~/aluminium/ruby/build/target/product/security/platform.x509.pem -inkey  
platform.key -out platform.p12 -name AndroidDebugKey -password  
pass:android  
  
# Convert  
keytool -importkeystore -  
-deststorepass android -  
-destkeypass android -  
-destkeystore platform.keystore -  
-srkeystore platform.p12 -  
-srestoretype PKCS12 -  
-srestorepass android -  
-alias AndroidDebugKey
```

Directly copy:

```
/home/$USER/aluminium/ruby/vendor/google/certs/devkeys/platform.keystore
```

Tips: For Virtual Device {Debug and test only}

Shell

```
keytool -genkey -v -keystore ~/.android/debug.keystore \  
-storepass android -alias androiddebugkey -keypass android \  
-keyalg RSA -keysize 2048 -validity 10000 \  
-dname "CN=Android Debug,O=Android,C=US"
```

Setup properties for build

Setup properties for build script

Add

\$ARSP_ROOT/vendor/google_shared/packages/desktop/Factory/factory/local.properties with the following lines under

Shell

```
# sdk.dir={**absoulute path** to your sdk directory} // usually  
~/Android/Sdk, this should match the path you entered when you install SDK  
in step 2  
# keystorePath={**absoulute path** to your key} // this should the same  
key that signs the rest of the Android platform  
  
# sdk.dir=/home/huanhuanl/Android/Sdk  
# keystorePath=/home/huanhuanl/.android/debug.keystore  
keystorePath=/home/huanhuanl/aluminium/ruby/vendor/google/certs/devkeys/pl  
atform.keystore
```

Build and deploy apk

If you have followed the step above to connect the DUT with ADB, you should see your device as an option to deploy the app. After selecting the device, you can just click the “Run ‘app’” button, then the IDE will build the debug version apk and deploy it to the device.

Tips: Need build test image and install to DUT

Build and Run without IDEs

Shell

```
# Quick debug cycle  
./gradlew assembleDebug  
  
# For production
```

```
./gradlew assembleRelease
```

Tips: Some Useful commands

Shell

```
# adb install -d -g -t app/build/outputs/apk/debug/app-debug.apk
```

```
# Connect
```

```
adb connect 192.168.0.14
```

```
# Install
```

```
adb -s 192.168.0.14 install -r -d -g -t  
app/build/outputs/apk/debug/app-debug.apk
```

```
adb -s 192.168.0.14 shell am start com.google.android.factory.factory
```

```
# Check Installed Path
```

```
adb shell pm path com.google.android.factory.factory
```

Factory Framework

Test List

The factory test list framework is a highly structured, data-driven system built on Protocol Buffers and Jetpack Compose.

Framework Overview

The factory test list framework utilizes `.txtpb` (Text Format Protocol Buffer) files to define a hierarchical structure. These definitions describe a tree of `TestObjects`, which are categorized as either Groups (containers) or Actions (executable test steps).

Data-Driven Design: Core logic is configured within the `.txtpb` via `args` (arguments). This allows for efficient code reuse by applying different parameters to existing behaviors.

Separation of Concerns: The architecture maintains a clean split between test logic, which resides in `ViewModel` classes, and the UI, which is handled by `Screen` (Compose) functions. These components are integrated through a central registry.

Layout & Key Files

The codebase is organized as follows:

Test List Definitions:

Location: `.../factory/app/src/main/test_lists/`

Files: `demo_google.txtpb`, `ruby_main.txtpb`, etc. These define what to run.

Action Implementation:

Location:

`.../factory/app/src/main/java/com/google/android/factory/factory/actions/`

Structure: Usually grouped by feature (e.g., `actions/camera/`, `actions/battery/`).

Files:

*`ViewModel.kt`: Contains the test logic (running shell commands, checking sensors).

*`Screen.kt`: The Compose UI displayed during the test.

*`Args.kt`: Defines the arguments the test can accept (mapped from the `.txtpb`).

The Registry (The Glue):

`ActionAggregator.kt`: This is the most important file. It contains a map that links the `action_name` string from the `.txtpb` to the actual `ViewModel` and `Screen` code.

How it Works (Execution Flow)

Loading: The `TestListUseCase` handles the loading and parsing of the selected `.txtpb` into an internal tree of `AnalyzedAction` objects.

Running: The `ActionTreeRunner` is responsible for traversing this tree structure.

Resolution: Upon reaching an action (e.g., `action_name: "Camera"`):

- It performs a lookup for "Camera" within `ActionAggregator.actionBuilders`.
- It utilizes the `ActionBuilder` to instantiate the `CameraViewModel` and locate the associated `CameraScreen`.

UI/Logic Link: The framework facilitates the connection by providing the `ViewModel` to the `Screen`. The `ActionTreeRunner` then invokes `viewModel.runAction(...)` to initiate the test sequence.

How to Add a New Test Action

Follow these steps to integrate a new action (e.g., `MyNewTest`):

Step 1: Define Arguments (Optional)

If the test requires specific parameters—such as a timeout or threshold—define them within an `Args` data class in the action's package.

Kotlin

```
data class MyNewTestArgs(  
    val timeoutSecs: Int = 10,  
    val threshold: Float = 0.5f  
)
```

Step 2: Create the ViewModel

Extend the `FactoryActionViewModel` and implement the `runAction` method.

Kotlin

```
class MyNewTestViewModel : FactoryActionViewModel<MyNewTestArgs>() {  
    override suspend fun runAction(action: AnalyzedAction.Action, args:  
MyNewTestArgs): FactoryActionResult {  
        // Implement your test logic here  
        // Return FactoryActionResult.Success or Failure  
        return FactoryActionResult.Success  
    }  
}
```

Step 3: Create the Screen (UI)

Define a `@Composable` function to handle the test status visualization or user interaction.

Kotlin

```
@Composable  
fun MyNewTestScreen(viewModel: MyNewTestViewModel) {  
    // Use Compose to draw your UI  
    Text("Running My New Test...")  
}
```

Step 4: Register in ActionAggregator.kt

Append the new action to the `actionBuilders` map:

Kotlin

```
"MyNewTest" to createActionBuilder(  
    MyNewTestArgs_GeneratedConverter(),           // Generated from your Args  
class  
    : MyNewTestArgs_GeneratedArgsEditor,         // Generated for the UI  
editor
```

```

        ::MyNewTestScreen,                // Your Screen
        ::MyNewTestViewModel,            // Your ViewModel
    ),

```

Step 5: Use it in a .txtpb file

Finally, include the action in the relevant test list definition:

```

Protobuf
subtests {
  label: "Run My New Test"
  action_name: "MyNewTest"
  args {
    fields {
      key: "timeoutSecs"
      value { number_value: 30 }
    }
  }
}
}

```

Summary of the Framework Logic

Component	Responsibility
.txtpb	Configuration: Defines the sequence and parameters.
AnalyzedAction	Structure: Parses the proto into a runnable tree.
ActionAggregator	Registry: Maps "String Name" $\rightarrow$ "Kotlin Code".
ViewModel	Logic: Performs the actual measurement/test.
ActionTreeRunner	Engine: Manages life-cycle, retries, and failures.

Example

Using `NuggetLightbar` as a concrete case study, the following guide details the iterative process for integrating a new test into the framework.

Step 1: Define Test Arguments (Args)

Within `NuggetLightbarViewModel.kt`, the `NuggetLightbarArgs` data class establishes the configurable parameters for the `.txtpb` file. The `@GenerateTestArgsUtils` annotation is utilized to automate the generation of necessary utility code.

```
Kotlin
@GenerateTestArgsUtils
data class NuggetLightbarArgs(
    // These fields map to args.fields in the .txtpb file
    val testColors: List<String> = listOf("red", "green", "blue", "white",
    "yellow", "magenta")
)
```

Step 2: Implement the Logic (ViewModel)

The `ViewModel` manages the core execution. `NuggetLightbarViewModel` extends `FactoryActionViewModel` and provides the implementation for `runActionImpl`.

Command Execution: Leverages

`factoryContext.adbClient.runShellCommand` to trigger system-level utilities such as `ectool`.

State Management: Utilizes a `StateFlow` (e.g., `uiState`) to communicate real-time progress or status updates to the UI layer.

Action Result: Dispatches a `FactoryActionResult.Success` or `Failure` notification back to the test runner.

```
Kotlin
class NuggetLightbarViewModel :
    FactoryActionViewModel<NuggetLightbarArgs>() {
    override suspend fun runActionImpl(): FactoryActionResult {
        // 1. Run a command to set the LED color
        val result = factoryContext.adbClient.runShellCommand("ectool led
        lightbar red=100")
    }
}
```

```

        // 2. Implementation logic (Wait for user input, verify, etc.)
        // ...

        return FactoryActionResult.Success
    }
}

```

Step 3: Build the User Interface (Screen)

The visual layer is constructed in `NuggetLightbarScreen.kt` using Jetpack Compose, observing the `ViewModel` state to dynamically render components.

```

Kotlin
@Composable
fun NuggetLightbarScreen(viewModel: NuggetLightbarViewModel) {
    val state by viewModel.uiState.collectAsState()

    Column {
        Text(text = "Status: ${state.statusMessage}")
        // Show buttons, indicators, or instructions based on the state
    }
}

```

Step 4: Register the Action (Aggregator)

This serves as the "glue" linking the Kotlin implementation to the test list. Within `ActionAggregator.kt`, map the identifier string (e.g., `"NuggetLightbar"`) to the corresponding classes:

```

Kotlin
// ActionAggregator.kt
"NuggetLightbar" to createActionBuilder(
    NuggetLightbarArgs_GeneratedConverter(), // Generated converter for Args
    ::NuggetLightbarArgs_GeneratedArgsEditor, // Generated bridge for the UI
    editor
    ::NuggetLightbarScreen, // Your Compose UI
    ::NuggetLightbarViewModel, // Your Logic/ViewModel
),

```

Step 5: Add to a Test List (.txtpb)

Finally, incorporate the `action_name` into a configuration resource, such as `demo_google.txtpb`.

```
Protobuf
subtests {
  label: "Nugget Lightbar Blind Test"
  action_name: "NuggetLightbar"
  args {
    fields {
      key: "testColors"
      value {
        list_value {
          values { string_value: "red" }
          values { string_value: "green" }
          values { string_value: "blue" }
        }
      }
    }
  }
}
```

Summary of Responsibilities

Step	Component	Responsibility
1. Define Args	NuggetLightbarViewMode 1.kt	Specifies configurable parameters (e.g., colors, timeouts).
2. Logic	NuggetLightbarViewMode 1.kt	Executes the actual test (shell commands, checks sensors).
3. UI	NuggetLightbarScreen.k t	Renders visual feedback for the operator.
4. Register	ActionAggregator.kt	Links "NuggetLightbar" to the Kotlin implementation.
5. Configure	*.txtpb	Integrates the test into the active device sequence.

Factory/MES Process

The ShopFloor system represents a foundational component of the manufacturing landscape, facilitating production line management, device metadata tracking (such as serial numbers and hardware IDs), and the reporting of telemetry to a centralized database. Within the AL architecture, this is realized as a robust gRPC service client.

How it Works

The implementation residing in `ShopFloorServiceViewModel.kt` functions as a critical intermediary, bridging the Android Factory Application with the remote Manufacturing Execution System (MES) server.

A. Communication (gRPC)

The application leverages gRPC (Remote Procedure Call) protocols for server-side interaction. When a `ShopFloorServiceView` action is invoked, the following sequence occurs:

Stub Creation: Establishes a secure communication channel utilizing the `serverUrl` defined within the factory configuration.

Method Mapping: Correlates the method identifier specified in the `.txtpb` configuration with a designated gRPC service call.

Data Exchange: Transmits the current `DeviceData` (local state) to the server, subsequently ingesting updated parameters in the response.

B. Primary Methods

GetDeviceInfo: Retrieves essential manufacturing metadata (e.g., component specifications) from the server.

NotifyStart / NotifyEnd: Communicates the lifecycle status of a device as it enters or departs a production station.

UpdateTestResult: Persists the final status of test execution into the centralized manufacturing database.

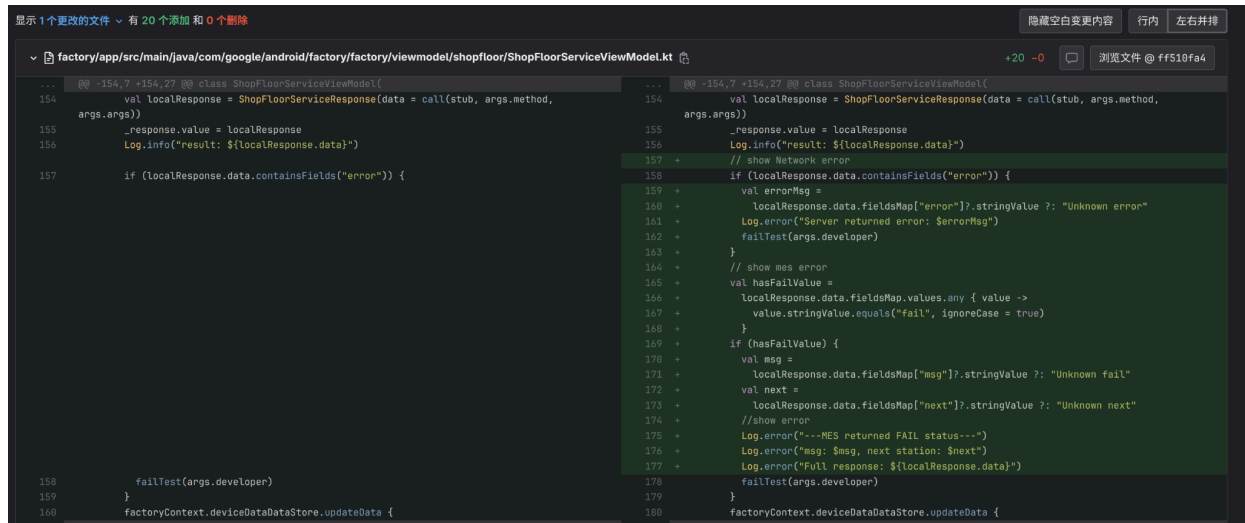
ActivateRegCode: Facilitates device registration with Google ecosystem services (incorporating ubind, gbind, and hwid procedures).

C. Data Sync

A pivotal characteristic of the architecture is that every successful ShopFloor transaction triggers an automatic update to the local `DeviceData` cache. This ensures

that any server-provided attributes, such as new serial numbers or configuration flags, are immediately available to subsequent test actions.

Example: Add error handling



```
... @@ -154,7 +154,27 @@ class ShopFloorServiceViewModel {
154     val localResponse = ShopFloorServiceResponse(data = call(stub, args.method,
    args.args))
155     _response.value = localResponse
156     Log.info("result: ${localResponse.data}")
157     if (localResponse.data.containsFields("error")) {
158         failTest(args.developer)
159     }
160     factoryContext.deviceDataDataStore.updateData {
161         ...
162         val localResponse = ShopFloorServiceResponse(data = call(stub, args.method,
163         args.args))
164         _response.value = localResponse
165         Log.info("result: ${localResponse.data}")
166         // show Network error
167         if (localResponse.data.containsFields("error")) {
168             val errorMsg =
169             localResponse.data.fieldsMap["error"]?.stringValue ?: "Unknown error"
170             Log.error("Server returned error: $errorMsg")
171             failTest(args.developer)
172         }
173         // show msg error
174         val hasFailValue =
175         localResponse.data.fieldsMap.values.any { value ->
176             value.stringValue.equals("fail", ignoreCase = true)
177         }
178         if (hasFailValue) {
179             val msg =
180             localResponse.data.fieldsMap["msg"]?.stringValue ?: "Unknown fail"
181             val next =
182             localResponse.data.fieldsMap["next"]?.stringValue ?: "Unknown next"
183             //show error
184             Log.error("---MES returned FAIL status---")
185             Log.error("msg: $msg, next station: $next")
186             Log.error("Full response: ${localResponse.data}")
187             failTest(args.developer)
188         }
189         factoryContext.deviceDataDataStore.updateData {
190             ...
191             failTest(args.developer)
192         }
193     }
194     factoryContext.deviceDataDataStore.updateData {
195         ...
196     }
197 }
```

Shopfloor

Software Overall Factory Process

SMT (Surface Mount Technology)
FATP (Final Assembly, Test, and Pack)
Example Station Configuration -TBD

Debug Tips

To verify the current status of the service port, execute the following:

```
Shell
sudo netstat -anulp | grep port | grep LISTEN
```

Terminate a specific process by port:

```
Shell
sudo kill -9 port
```

Monitor the ShopFloor service process execution:

Method 1: Background execution

```
Shell
nohup command &
# Relevant output is persisted to the nohup.out file
```

Method 2: Redirection to log file

Shell

```
python3 shopfloor_serveice.py >> logfile 2>&1
```

Troubleshooting: How do I resolve the "Too many open files" error?

Troubleshooting: How do I resolve the "Too many open files" error? — Server Management Tool

How to Expand ShopFloor

To integrate new ShopFloor capabilities—such as novel station validations or specialized data retrieval—adhere to the following procedure:

Step 1: Update the Protocol (Proto)

Verify that the `.proto` definition (typically `shopfloor.proto`) incorporates the new API. This step is necessary to generate the required Java/Kotlin Request and Response objects.

Step 2: Add to the ShopFloorMethod Enum

Within `ShopFloorServiceViewModel.kt`, extend the enum to include the new method, making it available for selection in test lists:

Kotlin

```
enum class ShopFloorMethod {  
    ...  
    MyNewMethod, // Register the new capability  
}
```

Step 3: Implement the Call Logic

Modify the `call()` function in the `ViewModel` to handle the new enum variant. This involves preparing the request payload and awaiting the gRPC response.

Kotlin

```
ShopFloorMethod.MyNewMethod -> {  
    val request = MyNewRequest.newBuilder()  
        .setData(getFactoryDeviceData())  
        .setCustomField(args[0])  
        .build()
```

```

val response = shopFloorGrpcStub.myNewMethod(request).await()
response.data // Return the parsed Struct data
}

```

Step 4: Use it in .txtpb

The expansion can now be referenced directly within any test list definition:

```

Protobuf
subtests {
  label: "Verify Custom Data"
  action_name: "ShopFloorServiceView"
  args {
    fields {
      key: "method"
      value { string_value: "MyNewMethod" }
    }
    fields {
      key: "args"
      value { list_value { values { string_value: "some_parameter" } } }
    }
  }
}
}

```

Summary of Workflow

Server side: Confirm the MES server is configured to support the new command.

App Proxy: Update `ShopFloorServiceViewModel` to encapsulate the gRPC logic.

Configuration: Integrate the new step into your `.txtpb` test sequence.

Hardware Identification

The `mapping_table.txtpb` serves as the primary configuration resource for Hardware Identification (HWID). It functions as a comprehensive database and rules engine, enabling the system to recognize and validate internal hardware components.

Process Explanation

1. Component Database

This file encompasses an extensive dictionary of hardware components, including Cameras, Storage, and RAM. Each entry specifies:

Component ID: A unique system identifier.

Probing Attributes: Technical identifiers such as USB VID/PID, PCI IDs, or model strings utilized for hardware discovery.

Approval Status: Indicates support across specific product milestones (e.g., PVT, MP).

2. Encoding Rules

The table defines the logic for generating the HWID string—a unique code representing the hardware configuration. It determines whether a specific part number or a component's presence is encoded into the final string.

3. Probing Process

During execution of the factory test sequence:

The system performs scans across hardware buses, including USB, PCI, and I2C.

Bus-level identifiers are matched against definitions in the [mapping_table](#).

The system resolves the precise manufacturer and part number for every detected component.

4. Patching & Side-loading

Through [HwDescUtils.kt](#), the system supports runtime "patching" of the table. This facility allows engineers to introduce support for new components, such as second-source hardware, via localized patch files without requiring a full system image rebuild.

Firmware

Adb/Fastboot Debug Cable Setup

```
yu@c:~/Downloads/partner_branch$ adb version 1.確認adb version為36.0以上
Android Debug Bridge version 1.0.41
Version 36.0.0-13206524
Installed as /opt/platform-tools/adb
Running on Linux 6.8.0-90-generic (x86_64)
yu@c:~/Downloads/partner_branch$ ls /etc/udev/rules.d/
51-android.rules 70-snap.firefox.rules 70-snap.marktext.rules 70-snap.scrpy.rules
yu@c:~/Downloads/partner_branch$ cat /etc/udev/rules.d/51-android.rules
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", MODE="0666", GROUP="plugdev"
SUBSYSTEM=="usb", ATTR{idVendor}=="04e8", MODE="0666", GROUP="plugdev"
SUBSYSTEM=="usb", ATTR{idVendor}=="0fce", MODE="0666", GROUP="plugdev"
SUBSYSTEM=="usb", ATTR{idVendor}=="0bb4", MODE="0666", GROUP="plugdev"
SUBSYSTEM=="usb", ATTR{idVendor}=="1004", MODE="0666", GROUP="plugdev"
SUBSYSTEM=="usb", ATTR{idVendor}=="22b8", MODE="0666", GROUP="plugdev"
SUBSYSTEM=="usb", ATTR{idVendor}=="2717", MODE="0666", GROUP="plugdev"
SUBSYSTEM=="usb", ATTR{idVendor}=="2a70", MODE="0666", GROUP="plugdev"
```

2.確認 /etc/udev/rules.d/下是否有51-android.rules, 並確認其內容

Install usbutils

```
Shell
sudo apt update
sudo apt install usbutils
```

Configure udev rules

```
Shell
sudo vim /etc/udev/rules.d/51-android.rules
```

Append

```
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", MODE="0666", GROUP="plugdev"
SUBSYSTEM=="usb", ATTR{idVendor}=="05c6", MODE="0666", GROUP="plugdev"
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="27a7",
MODE="0666", GROUP="plugdev"
```

Load and Restart

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Tips

None

https://arsp.googleusercontent.com/platform/vendor/google_shared/desktop/certs

Git For LCFC

Internal Gitlab

Setup Internal Gitlab

Follow install guide install and setup

<https://docs.gitlab.com/install/package/debian/>

Keep ARSP and internal Git sync

Push base code to rdgit - Admin Only

Shell

```
# ARSP factory, for example
```

```
~/aluminium/ruby/vendor/google_shared/packages/desktop/Factory
```

```
# add rdgit remote
```

```
git remote add rdgit
```

```
https://rdgit.lcfuturecenter.com/nbdtp\_de-sie/aluminium/ruby/Factory.git
```

```
# check remote branch
```

```
git remote -vv
```

```
# check current branch
```

```
git branch -vv
```

```
# push HEAD to rdgit main, Admin only, first setup
```

```
# git push rdgit HEAD:refs/heads/main
```

```
git push rdgit arsp/main
```

```
# Now you can see the base repo in Gitlab
```

Use Local Branch (Recommand)

Admin

We will use master branch for the internal develop

Shell

```
# Create local branch master for tracking rdgit
git checkout -b master rdgit/main

# Update
git pull arsp      # Pull updates from arsp/main

# link local branch and remote branch
# git branch --set-upstream-to=rdgit/main master

# Push to internal_main branch
# git push remote-branch local-branch
git push rdgit master
# Note, the master is enable protect branch, you can not push the changes
directly, please create you own's branch then do merge request step.
```

Developer

Shell

```
cd ~/aluminium/ruby/vendor/google_shared/packages/desktop/Factory

# git clone
https://rdgit.lcfuturecenter.com/nbdtp\_de-sie/aluminium/ruby/internal/factory-source.git

git remote add rdgit
https://rdgit.lcfuturecenter.com/nbdtp\_de-sie/aluminium/ruby/Factory.git

git pull master

git checkout master

# Create local branch f-dev for tracking rdgit master
git checkout -b f-dev master

# link local branch and remote branch
git branch --set-upstream-to=rdgit/master f-dev

# Push to remote branch
# git push remote-branch local-branch
```

```
git push rdgit f-dev
```

Upstream Sync {Sync base code from Google and Update}

Shell

```
git remote add rdgit  
https://rdgit.lcfuturecenter.com/nbdtp\_de-sie/aluminium/ruby/internal/factory-source.git
```

```
# 检出到 arsp/main && 拉取更新 && 推送到远程 rdgit 本地分支: 目标远程分支  
git checkout arsp/main && git pull arsp main && git push rdgit main:main
```

```
# git push rdgit develop:refs/heads/develop
```

Common Tips

Disable LidSwitch On debian 13 with Lenovo thinkpad P16s

Edit the systemd logind configuration

Shell

```
# Open the configuration file  
sudo vim /etc/systemd/logind.conf  
# Look for these lines (they may be commented out with #):  
#HandleLidSwitch=suspend  
#HandleLidSwitchDocked=ignore  
# Change them to:  
HandleLidSwitch=ignore  
HandleLidSwitchDocked=ignore  
HandleLidSwitchExternalPower=ignore
```

Restart the systemd-logind service

Shell

```
# Apply changes without rebooting:  
sudo reboot
```

Verify the changes

Close the lid and check if the system stays awake.

AG vs ARSP

Repo Diff

From the AG build (off Googleplex-Android), visit https://ci.android.com/builds/branches/git_desktop-trunk-release/grid?legacy=1 then find ruby.

From the ARSP build (off Partner-Android), visit <https://ci.android.com/> then input arsp-main as branch name.

Adb Switch User

Shell

```
$ adb shell pm list users
```

Users:

```
    UserInfo{0:Login Screen:811} running
```

```
    UserInfo{10:User:412} running
```

```
$ adb shell am get-current-user
```

```
10
```

```
$ adb shell am switch-user 10
```

```
adb shell am start com.google.android.factory.factory
```

Check keyboard mapping

```
# list keyname
adb shell getevent -l
# show all keys
adb shell dumpsys input
```

Mappings

```
F3 = KEY_FULL_SCREEN
F4 = KEY_SCALE
F6 = KEY_BRIGHTNESSDOWN
F7 = KEY_BRIGHTNESSUP
F8 = KEY_KBDILLUMTOGGLE
F9 = KEY_MICMUTE
```

F13 = KEY_ACCESSIBILITY
F14 = KEY_DICTATE
G = KEY_LEFTMETA
Caps = KEY_INSERT

Internal Proxy

Setup Internal Proxy via Web

Install Chrome Web Ext

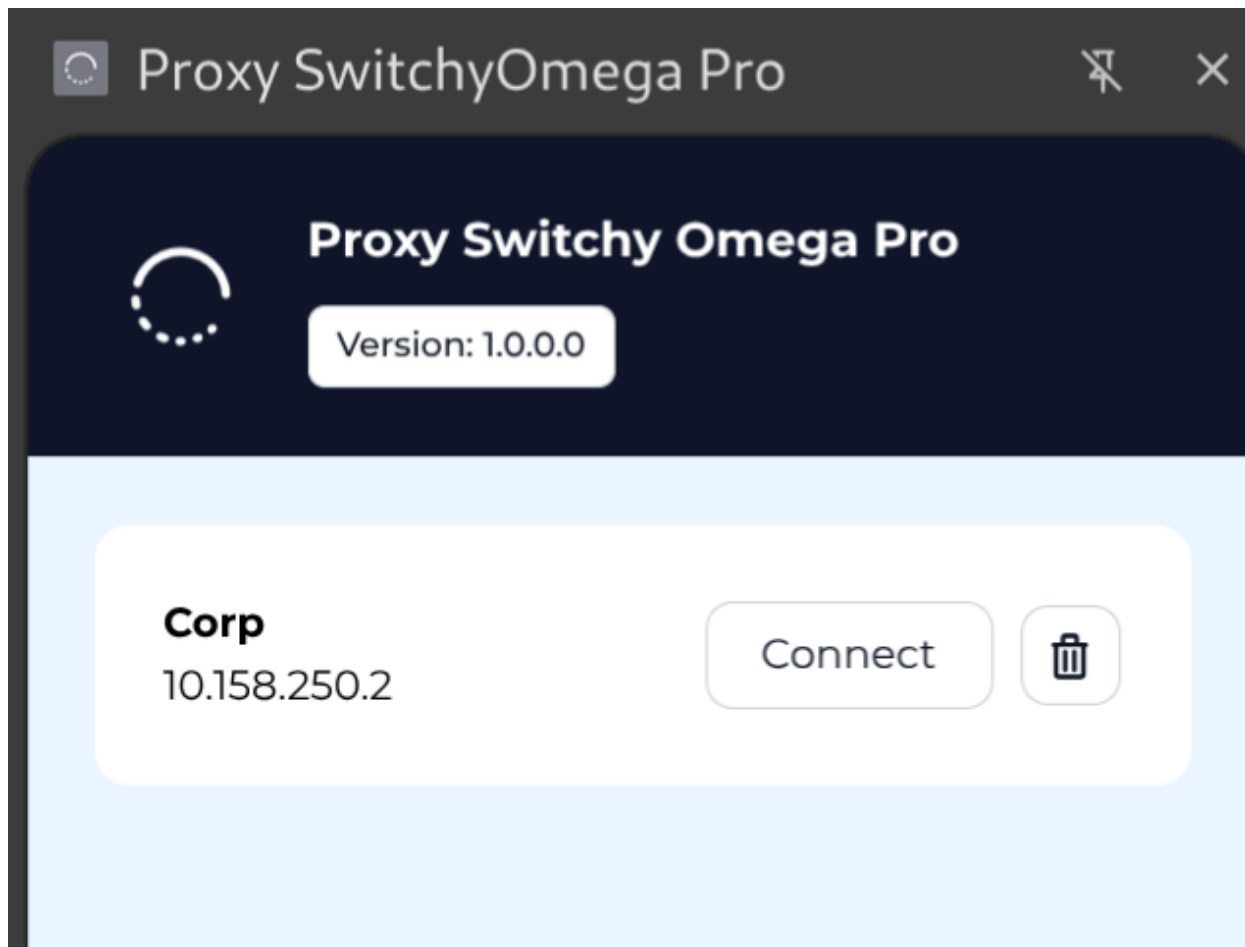
Open <https://chromewebstore.google.com/>

Search Proxy Switchy Omega Pro Proxy Switchy Omega Pro

Setup http proxy only: e.g: 10.158.250.2

Click connect Proxy Switchy Omega Pro

The web browser will auto detect proxy



Setup Internal Proxy via Terminal

Shell

```
# Set up env alias for quick connect
# Add this line to your .bashrc or .zshrc

# Proxy
# 公司代理快捷切换-包含认证信息的公司代理
alias pon='export http_proxy="http://username:password@10.158.250.2:80";
export https_proxy="http://username:password@0.158.250.2:80"; echo "Proxy
with Auth On"'
alias poff='unset http_proxy; unset https_proxy; echo "Proxy Off"'
# save and exit

# Enable proxy
pon
# Disable proxy
poff
```

BBR Activation for Linux

1. Configure BBR and FQ

Shell

```
echo "net.core.default_qdisc=fq" | sudo tee -a /etc/sysctl.d/99-bbr.conf
echo "net.ipv4.tcp_congestion_control=bbr" | sudo tee -a
/etc/sysctl.d/99-bbr.conf
```

2. Commit System Changes

Shell

```
sudo sysctl --system
```

3. Confirm Activation

Shell

```
sysctl net.ipv4.tcp_congestion_control
```

4. Verify BBR Kernel Module Loading

Shell

```
lsmod | grep bbr
```

Expected output: A result such as tcp_bbr 20480 14 indicates successful loading.

Useful Links:

- Issue Tracker: [Home - Issue Tracker](#)
- ARSP main: [arsp-main | Branch Grid](#)
- ARSP 26Q2 Desktop: [arsp-26Q2-desktop-release | Branch Grid](#)
- Stub Apk:

Open Questions

Any discussion point that is not fully fleshed out.

Document history

Google Docs has built-in document revision history: File > Version history > See version history, and you can name versions e.g., v1.0. However, readers must have EDIT

permission to see Docs' built-in history, so use this section if you want VIEWERS and COMMENTERS to see what's changed.